

CSCI 361 Lecture 15:

Reducibility

Shikha Singh

Announcements & Logistics

- Hand in **Exercise 12**
- No exercise for Tuesday, instead attempt Practice Midterm
 - Will be released today
 - Discuss on Tuesday
- Midterm 2 on Nov 6 (next Thurs) in class

Last Time

- Practice with reductions to prove a bunch of languages are undecidable
- Introduced mapping reducibility
 - Helps to reason about Turing (un)/recognizable languages

Today

- Rice's Theorem
- Using computational histories method and the PCPs to prove undecidability properties of CFG

Review: Exercise from Last Time

- Review the reductions from earlier:
 - From A_{TM} to E_{TM} : NO, in fact it is $\overline{A_{\text{TM}}} \leq_m E_{\text{TM}}$
 - From E_{TM} to EQ_{TM} : YES
 - From A_{TM} to $\text{REGULAR}_{\text{TM}}$: YES
- **Questions.**
 - Which of these are mapping reductions?
 - Is it possible to have a mapping reduction in all these cases?
 - Not always, let's prove this for E_{TM}

No Mapping Reduction: A_{TM} to E_{TM}

- Earlier reduction is mapping reduction from A_{TM} to $\overline{E_{\text{TM}}}$
 - That is, $\overline{A_{\text{TM}}} \leq_m E_{\text{TM}}$
- What can we say about E_{TM} ?
 - Since $\overline{A_{\text{TM}}}$ is not Turing recognizable, E_{TM} is also not Turing recognizable.
 - Found another example of a language that is not recognizable!
- **Question.** Is $\overline{E_{\text{TM}}}$ Turing recognizable?
- Can we prove there is no mapping reduction from A_{TM} to E_{TM} ?

No Mapping Reduction: A_{TM} to E_{TM}

Proof by contradiction.

- Suppose $A_{\text{TM}} \leq_m E_{\text{TM}}$ then $\overline{A_{\text{TM}}} \leq_m \overline{E_{\text{TM}}}$
- Since $\overline{E_{\text{TM}}}$ is Turing recognizable, this would imply $\overline{A_{\text{TM}}}$ is also Turing recognizable
- We know $\overline{A_{\text{TM}}}$ is not Turing recognizable $\Rightarrow \Leftarrow$

Undecidability Summary

Question. Which of these are decidable?

- Acceptance problems for DFA, CFG, TM
- Emptiness problems for DFA, CFG, TM
- Accepts all strings problem for DFA, CFG, TM
- Equivalence problems for DFA, CFG, TM

Rice's Theorem (Informal)

Any nontrivial **property of the languages** recognized by Turing machines is undecidable.

- Is the language empty? Is it finite? Is it infinite? Is it regular?
- Does the language contain strings in Σ^*
- Is the language the same as the language of another TM?

We proved many such examples in class.

HW 8 will have more practice with these.

Rice's Theorem

Any nontrivial **property of the languages** recognized by Turing machines is undecidable.

- Is the language empty? Is it finite? Is it infinite? Is it regular?
- Does the language contain strings in Σ^*
- Is the language the same as the language of another TM?

In contrast, questions about the TM's structure are decidable

- Has more than 15 states, has no transitions into its reject state, etc

Decidable or Not

Questions about behavior of TM's computation on inputs may or may not be decidable.

(HW 8 Problem): One of these is decidable, one is not:

- Does a given TM M and input w , does it ever move its head left when running on w ?
- Does a given TM M and input w , does it ever move its head three times in a row when running on w ?

Undecidable Languages about CFGs

The following languages about CFGs are all undecidable:

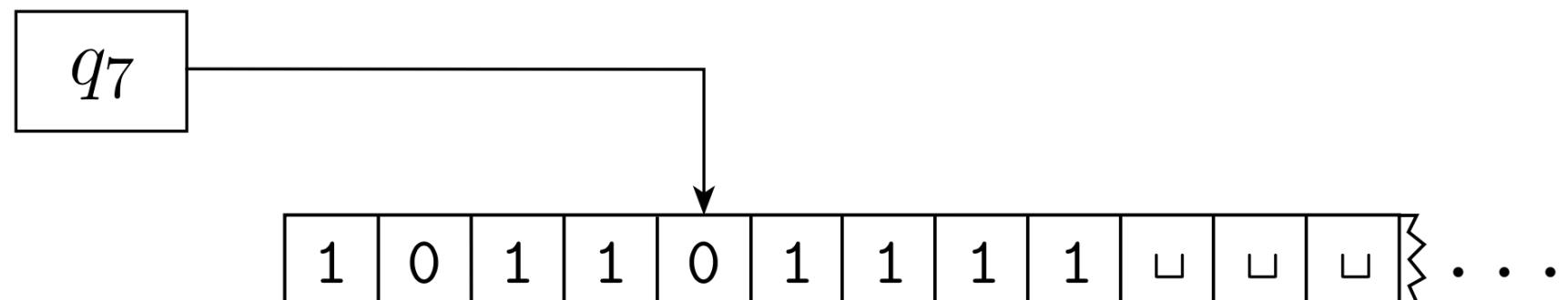
- **(All)** Given a CFG G , is $L(G) = \Sigma^*$?
- **(EQ)** Given two CFGs G_1, G_2 , is $L(G_1) = L(G_2)$?
- **(Disjoint)** Given two CFGs G_1, G_2 , is $L(G_1 \cap G_2) = \emptyset$?
- **(Ambiguity)** Given a CFG G , is it ambiguous?
- **(Disjoint Regular)** Given two CFGs G_1, G_2 , is $L(G_1 \cap G_2)$ a regular language?
- etc
- To prove undecidability, we need a way to encode the computation history of a Turing into "grammar form" or "string problem"

Recall: TM Configurations

- A configuration C_1 **yields** a configuration C_2 if the TM can legally go from C_1 to C_2 using its transition function δ
- Consider symbols $a, b, c \in \Gamma$ and strings $u, v \in \Gamma^*$ then

$ua q_i bv$ yields $u q_j acv$ if $\delta(q_i, b) = (q_j, c, L)$, and

$ua q_i bv$ yields $uac q_j v$ if $\delta(q_i, b) = (q_j, c, R)$



Computation Histories

- Consider a **deterministic** TM M and an input string w
- M 's computation on w can:
 - Halt and accept
 - Halt and reject
 - Or never halt (loop forever)
- For the first two cases, M 's **computation history** on w is a **finite** sequence $C_1, C_2, \dots, C_\ell$ where
 - C_1 is the start configuration
 - C_i yields C_{i+1} for each $1 \leq i \leq \ell - 1$
 - C_ℓ is an accept or reject configuration
- If M does not halt on w , no computation history exists

Post Correspondence Problem

- Simple problem about strings, defined by Emil Post (1946)
- Let Σ be any alphabet with at least two letters
- An instance of the Post correspondence problem (PCP) is given by a two sequences $A = (a_1, a_2, \dots, a_m)$ and $B = (b_1, b_2, \dots, b_m)$ where $a_i, b_i \in \Sigma^*$
- **Problem.** Does there exist a finite sequence $i_1, i_2, \dots, i_k$ where each i_j is an index from $1, \dots, m$ (with repetitions) such that

$$a_{i_1} a_{i_2} \dots a_{i_k} = b_{i_1} b_{i_2} \dots b_{i_k}$$

Post Correspondence Problem

- **Alternate Formulation:** An input is a collection of dominos with two sides: each containing two strings $\left[\frac{a_1}{b_1} \right], \left[\frac{a_2}{b_2} \right], \dots, \left[\frac{a_m}{b_m} \right]$
- Problem is to find a sequence of these dominoes (*repetitions are allowed*) such that the string formed by concatenating the top is the same as the string formed by concatenating the bottom

$$\left\{ \left[\frac{\mathbf{b}}{\mathbf{ca}} \right], \left[\frac{\mathbf{a}}{\mathbf{ab}} \right], \left[\frac{\mathbf{ca}}{\mathbf{a}} \right], \left[\frac{\mathbf{abc}}{\mathbf{c}} \right] \right\}.$$

- E.g. Consider

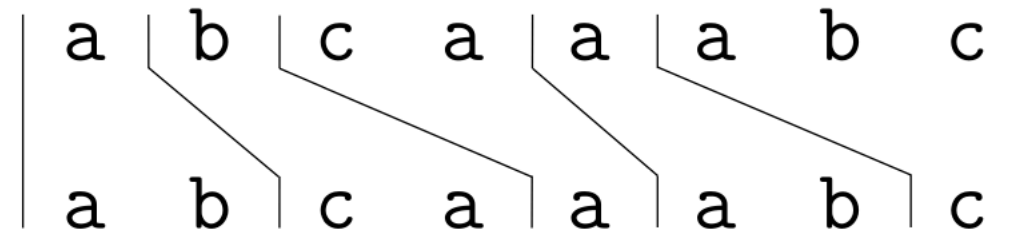
Post Correspondence Problem

- E.g. Consider

$$\left\{ \left[\frac{b}{ca} \right], \left[\frac{a}{ab} \right], \left[\frac{ca}{a} \right], \left[\frac{abc}{c} \right] \right\}.$$

- A possible solution

$$\left[\frac{a}{ab} \right] \left[\frac{b}{ca} \right] \left[\frac{ca}{a} \right] \left[\frac{a}{ab} \right] \left[\frac{abc}{c} \right]$$



Post Correspondence Problem

- E.g. Consider $\left\{ \left[\frac{abc}{ab} \right], \left[\frac{ca}{a} \right], \left[\frac{acc}{ba} \right] \right\}$
- No possible solution, why?
- Given a general instance, the PCP problem is to determine if it has a solution.
- **Theorem.** PCP is undecidable.

PCP is Undecidable

- **Reduce A_{TM} to PCP:** Given $\langle M, w \rangle$, create an instance of PCP such that it has a solution iff M accepts w
- **Key idea:** create dominoes such that a match between top and bottom strings forces M 's simulation on w

PCP is Undecidable

- Technicalities (ignore for now):
 - Assume M never moves its head off the left-hand end of the tape
 - If $w = \varepsilon$, assume $w = \sqcup$ in the construction
 - Modify PCP to **require starting with the first domino** $\left[\frac{t_1}{b_1} \right]$
- Can remove these restrictions at the end
-

PCP is Undecidable

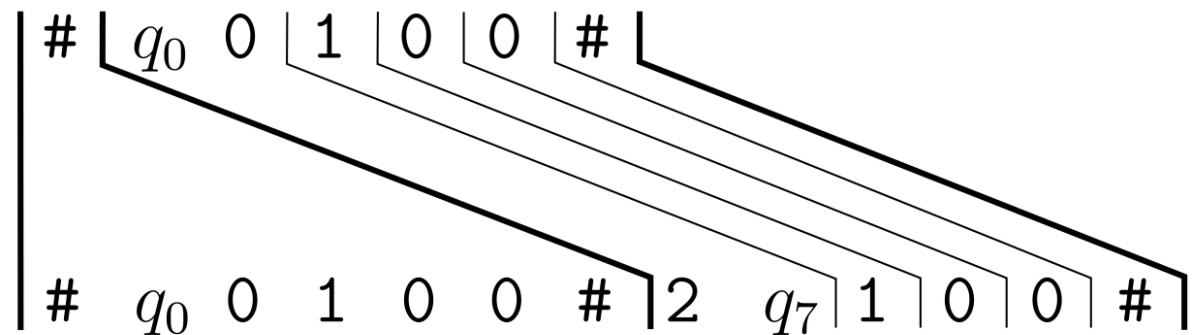
- **Part 1.** Let the first domino be $\left[\frac{\#}{\#w_1w_2\dots w_n\#} \right]$
- **Part 2.** For each transition of the type $\delta(q, a) = (r, b, R)$, create a domino $\left[\frac{qa}{br} \right]$ (*simulate moves adjacent to head position*)
- **Part 3.** Similar dominos to simulate for left moves.
- **Part 4.** To "copy over" symbols that are not adjacent to head position on either side, create domino $\left[\frac{a}{a} \right]$ for each $a \in \Gamma$
- **Part 5.** To match the $\#$ symbols in the middle and at end add $\left[\frac{\#}{\#} \right]$ and $\left[\frac{\#}{\sqcup \#} \right]$

Construction Example

- Consider an M that starts in q_0 on input 0100 and $q_0 0100$ yields $2q_7 100$ by following $\delta(q_0, 0) = (q_7, 2, R)$

- Part 1 adds the first domino
$$\left[\begin{array}{c} \# \\ \hline \#q_0 0100\# \end{array} \right]$$

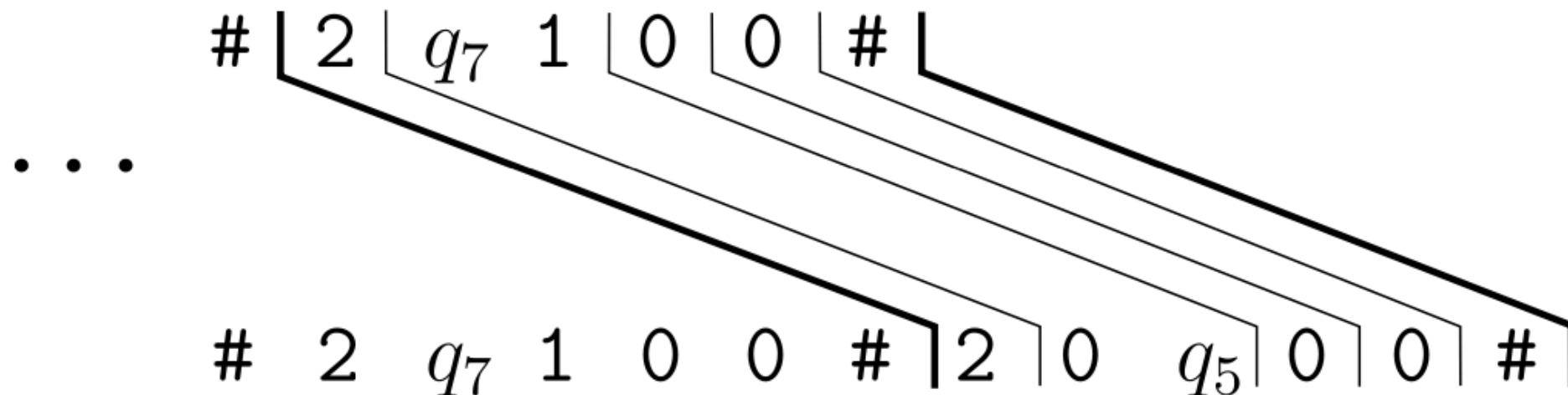
- Part 2 adds the domino
$$\left[\begin{array}{c} q_0 0 \\ \hline 2q_7 \end{array} \right]$$



- Using Part 5 can force the match:

Construction Example

- Recall that $q_0 0100$ yields $2q_7 100$
- Now suppose $\delta(q_7, 1) = (q_5, 0, R)$
- That is, $2q_7 100$ yields $20q_5 00$, then we add the domino $\left[\begin{array}{c} q_7 1 \\ 0 q_5 \end{array} \right]$
- Thus, the partial match looks like this:

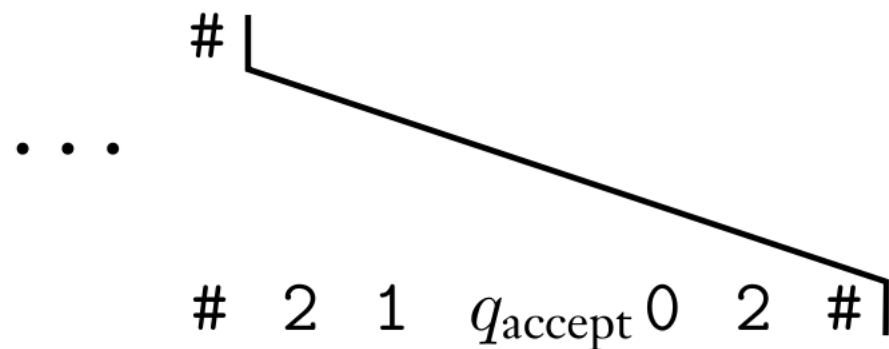


Construction Example

- Finally, to handle the last transition that includes a q_{accept} add

$$\left[\frac{a \ q_{\text{accept}}}{q_{\text{accept}}} \right] \text{ and } \left[\frac{q_{\text{accept}}a}{q_{\text{accept}}} \right] \text{ for each } a \in \Gamma$$

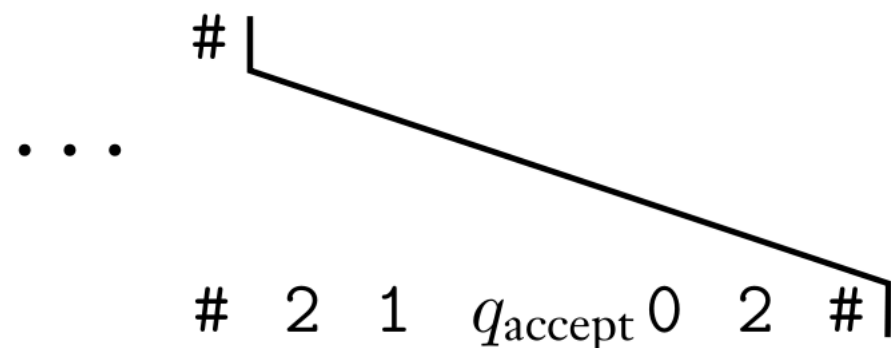
Suppose the last configuration of M is



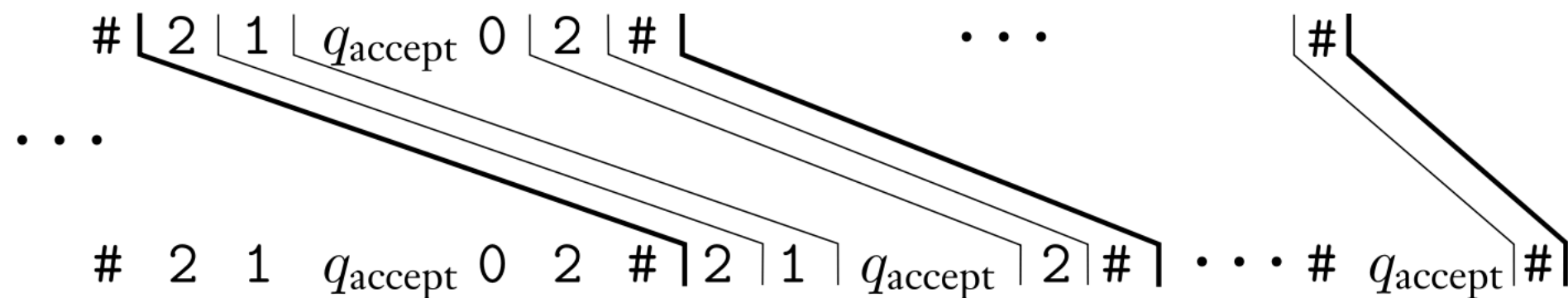
Then to allow the top to catch up we need these extra dominoes

Construction Example

- Suppose the last configuration of M is



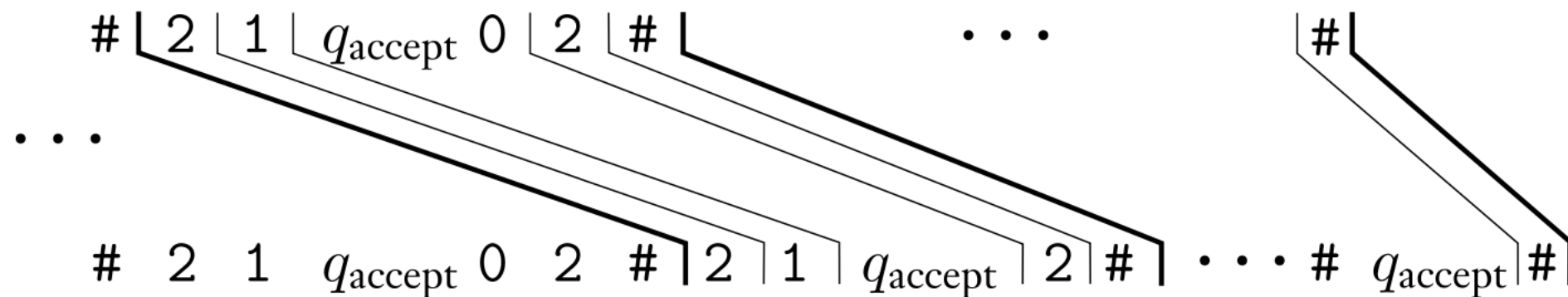
Then to allow the top to catch up we need these extra dominoes



Construction Example

- Finally, for the last step to match the extra $q_{\text{accept}}\#$ at the bottom

add the domino $\left[\frac{q_{\text{accept}}\#\#}{\#} \right]$



Questions

- Is PCP recognizable?
- Is $\overline{PCP}$ recognizable?

PCP to CFL Reductions

- Using PCP, we can show a bunch of questions about CFGs are undecidable:
 - **(Ambiguity)** Given a CFG G , is it ambiguous?
 - **(Disjoint)** Given two CFGs G_1, G_2 , is $L(G_1 \cap G_2) = \emptyset$?
 - **(Disjoint Regular)** Given two CFGs G_1, G_2 , is $L(G_1 \cap G_2)$ a regular language?
 - ... etc.

Reductions from PCP

- **Question.** Reduce PCP to show that

$\cap_{\text{CFG}} = \{ \langle G_1, G_2 \rangle \mid G_1, G_2 \text{ are CFGs such that } L(G_1) \cap L(G_2) = \emptyset \}$
is undecidable.

CFG Disjointness is Undecidable

Proof. (Cont.)

- Create CFLs L_A and L_B as follows:

$$A \rightarrow a_1 A i_1 \mid a_2 A i_2 \mid \cdots \mid a_m A i_m$$

$$A \rightarrow a_1 i_1 \mid a_2 i_2 \mid \cdots \mid a_m i_m$$

$$B \rightarrow b_1 B i_1 \mid b_2 B i_2 \mid \cdots \mid b_m B i_m$$

$$A \rightarrow a_1 i_1 \mid a_2 i_2 \mid \cdots \mid a_m i_m$$

Question. What can we say about the strings in $L(L_A) \cap L(L_B)$?

CFG Disjointness is Undecidable

- **Example.** $\left\{ \left[\frac{b}{ca} \right], \left[\frac{a}{ab} \right], \left[\frac{ca}{a} \right], \left[\frac{abc}{c} \right] \right\}.$

$A \rightarrow bA1 \mid aA2 \mid caA3 \mid abcA4 \mid b1 \mid a2 \mid ca3 \mid abc4$

$B \rightarrow caB1 \mid abB2 \mid aB3 \mid cB4 \mid ca1 \mid ab2 \mid a3 \mid c4$

CFG Disjointness is Undecidable

• **Example.** $\left\{ \left[\frac{b}{ca} \right], \left[\frac{a}{ab} \right], \left[\frac{ca}{a} \right], \left[\frac{abc}{c} \right] \right\}.$

$$A \rightarrow bA1 \mid aA2 \mid caA3 \mid abcA4 \mid b1 \mid a2 \mid ca3 \mid abc4$$

$$B \rightarrow caB1 \mid abB2 \mid aB3 \mid cB4 \mid ca1 \mid ab2 \mid a3 \mid c4$$

Solution to PCP:

$$\left[\frac{a}{ab} \right] \left[\frac{b}{ca} \right] \left[\frac{ca}{a} \right] \left[\frac{a}{ab} \right] \left[\frac{abc}{c} \right]$$

String derived from A : $a \ b \ ca \ a \ abc \ 42312$

String derived from B : $ab \ ca \ a \ ab \ c \ 42312$

CFG Disjointness is Undecidable

Proof. Create CFLs L_A and L_B as follows:

$$A \rightarrow a_1 A i_1 \mid a_2 A i_2 \mid \cdots \mid a_m A i_m$$

$$A \rightarrow a_1 i_1 \mid a_2 i_2 \mid \cdots \mid a_m i_m$$

$$B \rightarrow b_1 B i_1 \mid b_2 B i_2 \mid \cdots \mid b_m B i_m$$

$$A \rightarrow a_1 i_1 \mid a_2 i_2 \mid \cdots \mid a_m i_m$$

Question. What can we say about the strings in $L(L_A) \cap L(L_B)$?

Correspond to solutions to PCP instance.

Thus, intersection is empty iff PCP has no solution.

Thus, can decide PCP using a decider for $\cap_{CFG}$. $\Rightarrow \Leftarrow$

ALLCFG is undecidable

Lemma. Complement of languages L_A and L_B is a CFL.

One way to see this, is that the grammars for L_A and L_B can be recognized by deterministic push-down automata. Turns out the complement of DCFLs are CFLs. (Can flip accept/reject states similar to an NFA)

Question. Using this fact, we can complement $\overline{L_A}$ and $\overline{L_B}$ and note that $\overline{\overline{L_A} \cap \overline{L_B}} = \overline{L_A} \cup \overline{L_B}$.

Thus, $\overline{\overline{L_A} \cap \overline{L_B}} = \Sigma^*$ if and only if PCP has no solution.

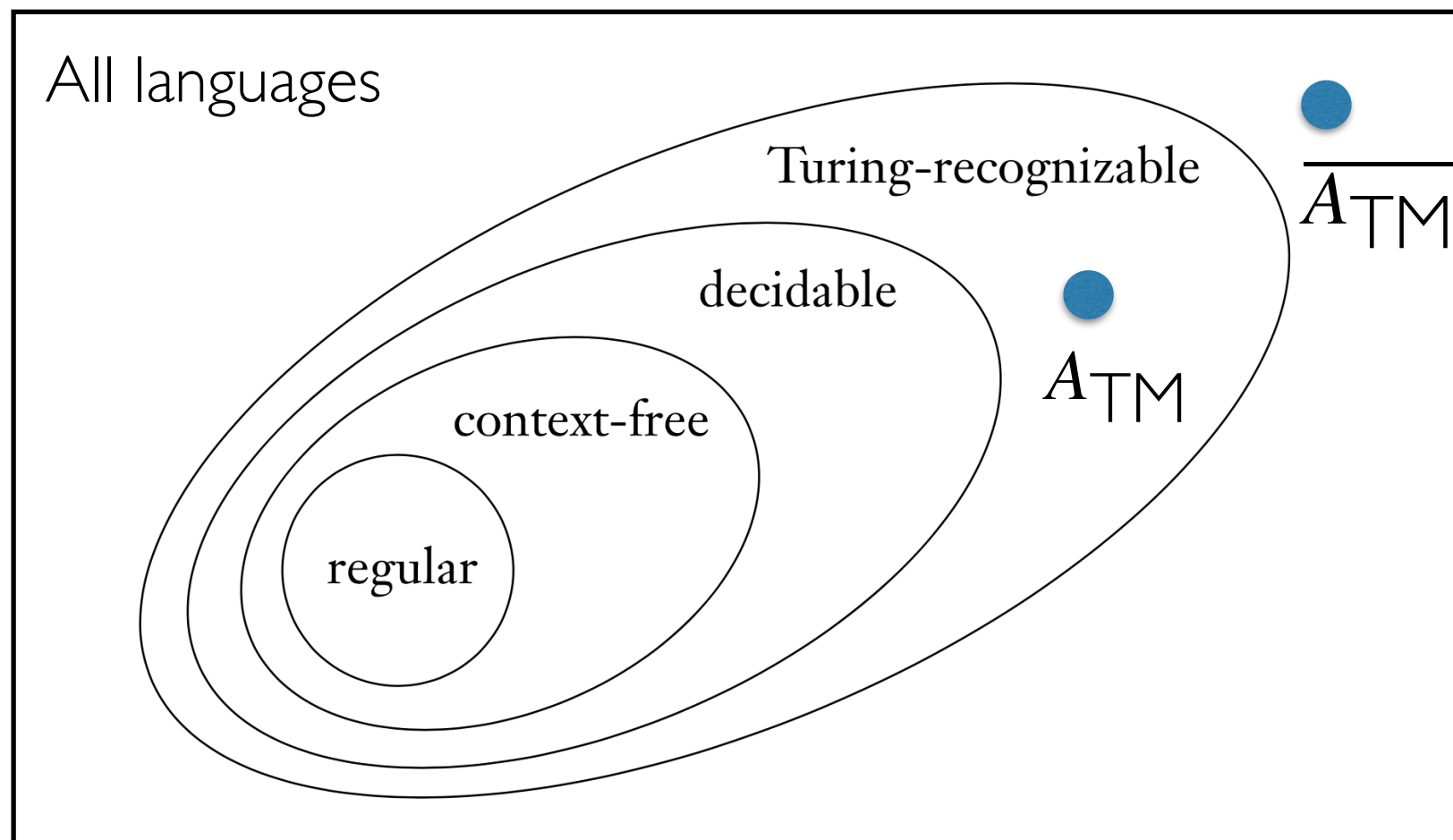
This proves that ALLCFG is also undecidable. ■

Undecidability Takeaways

- Almost all properties of regular languages are decidable
- Lots of undecidable problems about CFGs
 - Let G_1, G_2 be CFGs and R be a regular expression, then the following questions are undecidable:
 - Is $L(G_1) = L(G_2)$?
 - Is $L(G_1) = L(R)$?
 - Is $L(G_1) \subseteq L(G_2)$?
 - Is $L(R) \subseteq L(G_1)$?
- Deciding any non-trivial property of TM is undecidable
- This is a motivation for studying restricted models of computation

Our Picture

- **Final Question.** Is there a language L such that L is not Turing recognizable and $\bar{L}$ is also not Turing recognizable.
- **Recall.** If $A \leq_m B$ and A is not Turing recognizable, then B is not Turing recognizable.



Class Exercise

- **Theorem.** EQ_{TM} is neither Turing recognizable nor co-Turing recognizable (its complement is not Turing recognizable).
- **Proof outline.**
 - To show EQ_{TM} is not Turing recognizable, need to reduce a known Turing unrecognizable language to it
 - Show that $A_{TM} \leq_m EQ_{TM}$ and $A_{TM} \leq_m \overline{EQ_{TM}}$
 - How does this prove the theorem?
 - Mapping reductions are closed under complement!

Completed Picture of Computability

All Languages

